



INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA
DO CEARÁ - CAMPUS CRATO
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Relatório Técnico: Sistema Adaptativo para Seleção de Algoritmos de
Busca e Ordenação

Alunos:

Tecla Fernandes Oliveira

Gustavo Eliel de Figueiredo Silva

Fernando Barbosa da Silva

Lucas Oliveira de Souza

Equipe: Tropa
2026.1

RELATÓRIO TÉCNICO

Sistema Adaptativo para Seleção de Algoritmos de Busca e Ordenação

1. Introdução

A escolha adequada de algoritmos de busca e ordenação é um dos fatores que mais influenciam o desempenho de sistemas computacionais. Embora diversos algoritmos apresentem a mesma finalidade, suas características diferem significativamente quanto ao tempo de execução, consumo de memória, estabilidade e comportamento em diferentes cenários de dados.

Neste contexto, foi desenvolvido um Sistema Adaptativo de Seleção de Algoritmos capaz de recomendar automaticamente o algoritmo mais apropriado para uma determinada situação. O sistema analisa características do conjunto de dados e requisitos fornecidos pelo usuário, utilizando regras heurísticas para determinar qual algoritmo apresenta maior adequação ao problema. Além da recomendação, o sistema também realiza análises de complexidade computacional, benchmark de desempenho e validação empírica dos resultados, permitindo comparar teoria e prática de forma integrada.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um sistema adaptativo de recomendação de algoritmos capaz de analisar características de conjuntos de dados e requisitos de execução para selecionar automaticamente o algoritmo de busca ou ordenação mais adequado para cada cenário. O sistema busca combinar conceitos teóricos de complexidade computacional com análises práticas de desempenho, permitindo avaliar como fatores como tamanho do dataset, grau de ordenação, presença de elementos repetidos, restrições de memória e necessidade de estabilidade influenciam diretamente na escolha do algoritmo mais eficiente.

Além da recomendação automática, o projeto visa proporcionar uma ferramenta educacional que auxilie na compreensão do comportamento dos algoritmos clássicos de busca e

ordenação, demonstrando na prática as diferenças entre suas características teóricas e seus resultados experimentais.

2.2 Objetivos Específicos

Para atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

- Implementar algoritmos clássicos de ordenação, incluindo Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort e Heap Sort.
- Implementar algoritmos clássicos de busca, incluindo Busca Sequencial, Busca Binária e Busca Hash.
- Desenvolver um módulo de análise capaz de identificar automaticamente características relevantes do conjunto de dados, como tamanho, tipo, amplitude dos valores, percentual de duplicatas e grau de ordenação.
- Construir um mecanismo de inferência baseado em regras heurísticas para recomendar algoritmos de acordo com as características identificadas e os requisitos informados pelo usuário.
- Disponibilizar dois modos de operação: um modo baseado em questionário e um modo direto, no qual o usuário fornece o conjunto de dados para análise automática.
- Apresentar informações teóricas sobre cada algoritmo, incluindo complexidade de melhor caso, caso médio e pior caso, uso de memória, estabilidade e comportamento prático.
- Implementar mecanismos de instrumentação para medir métricas experimentais, como tempo de execução e consumo aproximado de memória.
- Realizar benchmark dos algoritmos implementados, permitindo comparar seus desempenhos em diferentes cenários de entrada.
- Desenvolver um processo de validação empírica para verificar a qualidade das recomendações produzidas pelo sistema.
- Comparar os resultados experimentais obtidos com as previsões teóricas de complexidade computacional, evidenciando a relação entre teoria e prática.
- Produzir relatórios e visualizações gráficas que auxiliem na interpretação dos resultados obtidos durante os experimentos.

3. Fundamentação Teórica

3.1 Algoritmos de Ordenação

Foram implementados os seguintes algoritmos:

Bubble Sort

Realiza trocas sucessivas entre elementos adjacentes até que a lista esteja ordenada.

Selection Sort

Seleciona repetidamente o menor elemento e o posiciona na região ordenada.

Insertion Sort

Insere cada elemento em sua posição correta dentro da porção já ordenada do vetor.

Merge Sort

Utiliza a estratégia de divisão e conquista, dividindo o problema em subproblemas menores.

Quick Sort

Seleciona um pivô e particiona o vetor em elementos menores e maiores que esse pivô.

Heap Sort

Utiliza uma estrutura Heap para realizar a ordenação de maneira eficiente.

3.2 Algoritmos de Busca

Busca Sequencial

Percorre os elementos um a um até encontrar o valor procurado.

Busca Binária

Realiza busca eficiente em conjuntos previamente ordenados.

Busca Hash

Utiliza tabela hash para permitir buscas com tempo médio constante.

4. Arquitetura do Sistema

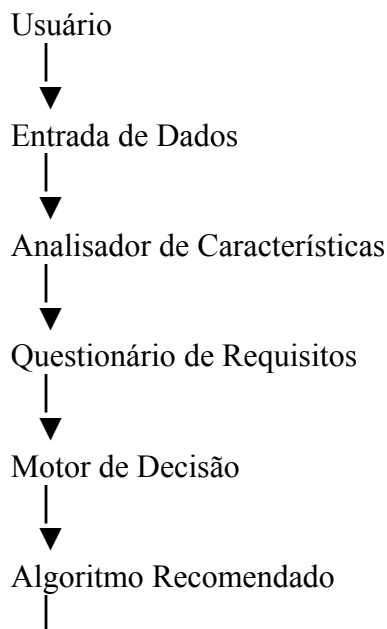
O sistema foi desenvolvido seguindo uma arquitetura modular, na qual cada componente possui responsabilidades bem definidas e independentes. Essa abordagem facilita a manutenção, reutilização de código, realização de testes e futuras expansões do projeto.

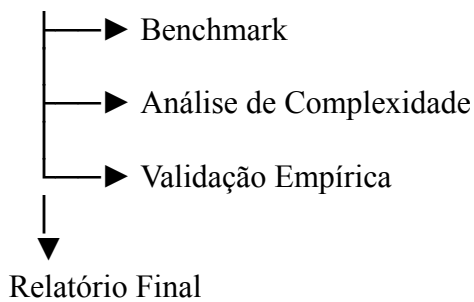
A arquitetura foi projetada para separar as etapas de coleta de informações, análise dos dados, tomada de decisão, validação experimental e apresentação dos resultados. Dessa forma, cada módulo atua de maneira especializada dentro do fluxo geral da aplicação.

O funcionamento do sistema pode ser resumido nas seguintes etapas:

1. Recebimento dos dados ou informações do usuário;
2. Análise das características do conjunto de dados;
3. Coleta de requisitos adicionais;
4. Aplicação das regras do motor de decisão;
5. Seleção do algoritmo mais adequado;
6. Exibição do relatório final;
7. Execução opcional de benchmarks e validações experimentais.

A estrutura abaixo representa o fluxo lógico do sistema:





4.1 Módulo de Entrada de Dados

O sistema oferece dois modos de operação para fornecer maior flexibilidade ao usuário.

Modo Questionário

Nesse modo, o usuário informa características gerais do problema através de perguntas estruturadas. As respostas permitem ao sistema identificar restrições e necessidades específicas da aplicação.

Entre as informações coletadas estão:

- quantidade de elementos;
- necessidade de estabilidade;
- existência de restrições de memória;
- presença de muitos elementos repetidos;
- objetivo da operação (busca ou ordenação).

Modo Direto

Nesse modo, o usuário fornece diretamente um vetor contendo os dados a serem analisados. A partir dessa entrada, o sistema calcula automaticamente diversas características do conjunto de dados, reduzindo a necessidade de intervenção manual e tornando a recomendação mais objetiva.

4.2 Módulo de Análise de Características

O módulo de análise é responsável por extrair informações relevantes do conjunto de dados que possam influenciar na escolha do algoritmo. As métricas analisadas incluem:

- tamanho do conjunto;
- tipo dos dados;
- amplitude dos valores;
- percentual de elementos duplicados;
- grau de ordenação.

Essas informações são utilizadas posteriormente pelo motor de decisão para determinar quais algoritmos são mais adequados para o cenário analisado.

Cálculo do Grau de Ordenação

O grau de ordenação é estimado através da contagem de inversões. Uma inversão ocorre quando:

$$i < j \text{ e } A[i] > A[j]$$

Quanto menor o número de inversões, mais próximo o conjunto está de uma sequência ordenada. Para evitar alto custo computacional em conjuntos muito grandes, o sistema utiliza amostragem aleatória para estimar o número de inversões, reduzindo significativamente o tempo de processamento sem comprometer a qualidade da análise.

4.3 Módulo de Coleta de Requisitos

Nem todas as informações necessárias para a escolha do algoritmo podem ser obtidas apenas analisando os dados. Por esse motivo, o sistema utiliza um módulo específico para coletar requisitos relacionados ao contexto de utilização. Entre os requisitos considerados estão:

- necessidade de estabilidade;
- restrições de memória;
- tipo da operação desejada;
- características operacionais do problema.

Essas informações complementam os dados obtidos pelo analisador e tornam a recomendação mais aderente ao cenário real.

4.4 Motor de Decisão

O motor de decisão constitui o núcleo do sistema. Seu objetivo é avaliar todas as alternativas disponíveis e atribuir uma pontuação para cada algoritmo com base em regras heurísticas previamente definidas. Essas regras foram construídas a partir das propriedades conhecidas dos algoritmos estudados na disciplina. Exemplos de regras utilizadas:

- bônus para Insertion Sort em conjuntos quase ordenados;
- penalização de algoritmos não estáveis quando a estabilidade é necessária;
- eliminação de algoritmos $O(n^2)$ em conjuntos muito grandes;
- priorização de algoritmos com menor consumo de memória quando existem restrições de recursos.

Ao final da avaliação, o algoritmo com maior pontuação é selecionado como recomendação principal.

4.5 Módulo de Benchmark

O módulo de benchmark é responsável por realizar medições experimentais dos algoritmos implementados. Seu principal objetivo é permitir a comparação entre os resultados teóricos previstos pela análise de complexidade e o comportamento observado durante a execução. As métricas atualmente coletadas incluem:

- tempo de execução;
- consumo aproximado de memória;
- memória de pico utilizada durante o processamento.

Essas informações auxiliam na validação das recomendações realizadas pelo sistema.

4.6 Módulo de Validação Empírica

O módulo de validação foi desenvolvido para verificar a qualidade das recomendações realizadas pelo motor de decisão. O processo consiste em executar todos os algoritmos aplicáveis a um mesmo cenário, medir seus tempos de execução e gerar um ranking baseado no

desempenho real obtido. Posteriormente, a recomendação do sistema é comparada ao ranking experimental.

A recomendação é considerada satisfatória quando o algoritmo sugerido encontra-se entre os dois algoritmos com melhor desempenho observado. Esse processo permite avaliar objetivamente a eficácia das heurísticas utilizadas pelo seletor adaptativo.

5. Análise de Complexidade

Este módulo centraliza informações teóricas sobre todos os algoritmos implementados. Para cada algoritmo são armazenados:

- melhor caso;
- caso médio;
- pior caso;
- complexidade espacial;
- estabilidade;
- comportamento prático.

Essas informações são utilizadas na geração dos relatórios e auxiliam o usuário na compreensão das características de cada algoritmo. O sistema mantém informações teóricas para todos os algoritmos implementados.

Exemplo:

Algoritmo	Melhor Caso	Caso Médio	Pior Caso
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

6. Instrumentação

O sistema realiza medições experimentais utilizando o módulo Benchmark. As métricas coletadas são:

Tempo de execução

Obtido através da função `perf_counter()`.

Uso aproximado de memória

Obtido utilizando o módulo `tracemalloc`. São armazenados:

- memória atual;
- memória de pico.

Essas métricas permitem comparar desempenho teórico e desempenho observado.

7. Validação Empírica

Para validar a qualidade da recomendação produzida pelo seletor, foi desenvolvido um processo de validação experimental. Para cada cenário:

1. O sistema gera um conjunto de dados;
2. Todos os algoritmos são executados;
3. Os tempos reais são medidos;
4. É criado um ranking experimental;
5. A recomendação do sistema é comparada ao ranking obtido.

Considera-se uma recomendação correta quando o algoritmo recomendado aparece entre os dois melhores algoritmos observados experimentalmente. A taxa de acerto é calculada por:

$$\text{Taxa de Acerto} = (\text{Acertos} / \text{Total de Cenários}) \times 100$$

8. Resultados Experimentais

Os experimentos realizados demonstraram que:

- algoritmos $O(n^2)$ tornam-se inviáveis para grandes conjuntos de dados;
- Insertion Sort apresenta excelente desempenho em dados quase ordenados;
- Merge Sort mantém comportamento consistente independentemente do cenário;
- Quick Sort apresenta excelente desempenho médio;
- Busca Hash é geralmente superior para grandes volumes de consultas;
- Busca Binária apresenta excelente desempenho quando os dados já estão ordenados.

Os resultados experimentais mostraram forte aderência às previsões teóricas de complexidade.

9. Estrutura do Projeto

O projeto encontra-se organizado em módulos especializados:

- algoritmos de busca;
- algoritmos de ordenação;

- análise de características;
- motor de decisão;
- benchmark;
- validação;
- testes automatizados.

Essa separação favorece a manutenção, reutilização e expansão futura.

10. Conclusão

O desenvolvimento deste projeto permitiu aplicar conceitos fundamentais de algoritmos e estruturas de dados em uma solução prática e interativa. O sistema foi capaz de combinar análise automática de características, coleta de requisitos e regras heurísticas para recomendar algoritmos adequados a diferentes cenários.

Além disso, a inclusão de benchmark, análise de complexidade e validação empírica permitiu comparar teoria e prática, evidenciando como diferentes características dos dados influenciam diretamente o desempenho dos algoritmos. Os resultados obtidos demonstram que a seleção adequada de algoritmos pode produzir ganhos significativos de desempenho, reforçando a importância do estudo de complexidade computacional e análise experimental.

11. Trabalhos Futuros

Como possíveis evoluções do projeto destacam-se:

- implementação de interface gráfica;
- armazenamento histórico das execuções;
- exportação automática de relatórios;
- inclusão de novos algoritmos;
- suporte a múltiplos tipos de dados;
- integração com bibliotecas de visualização avançada;
- utilização de aprendizado de máquina para aprimorar as recomendações.